

DEPARTMENT OF COMPUTER SCIENCE



University of
Reading

CS1IP: Imperative Programming

File I/O and Basic Collections

Learning Goals

- What is file I/O?
- How to do file I/O in Java?
- What Java Collections?

What is file I/O?

What is a program?

A sequence of **instructions** that is **executed** by a **computer**.

These instructions may required use stored data.

So, it requires gather input and output data outside the program.

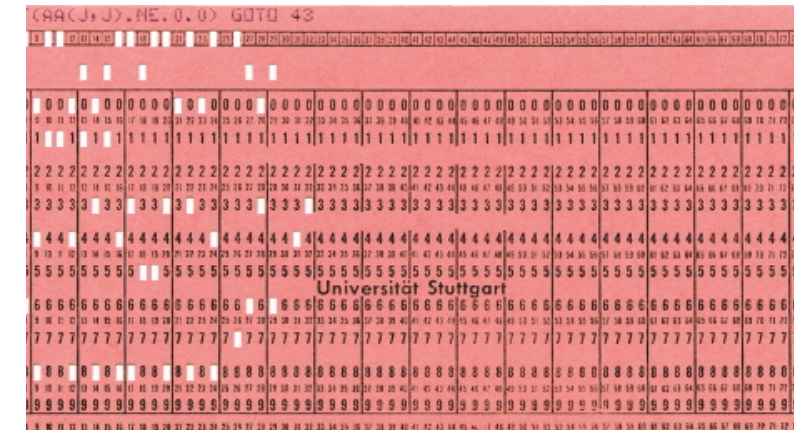
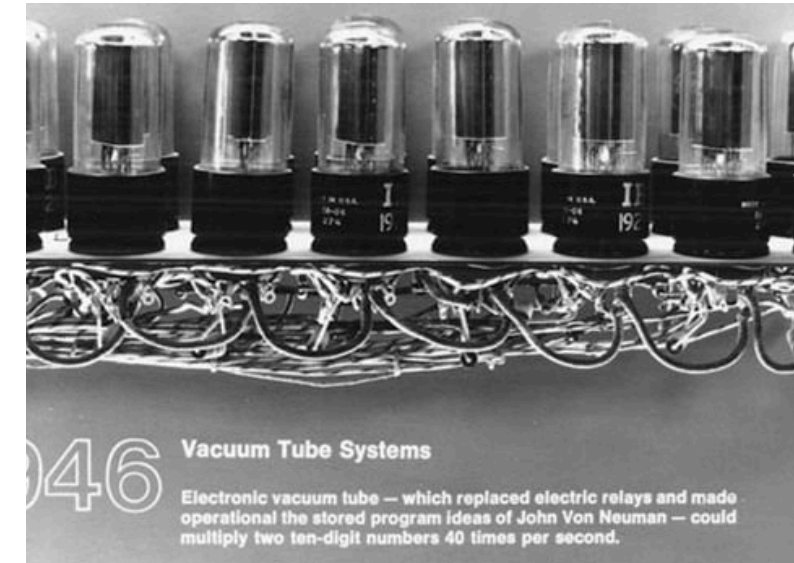
What is I/O?

- I/O Stands for Input and Output.
- In early computing have no file I/O

File I/O

Operations to read from and write to files.

- The first File I/O by old computer was "I/O cards"
- Today file as Data Persistence, i.e., used for long-term data retention.



File I/O in Java

Main classes for File I/O in Java

- The best way to represents file and directories today is using `Path` objects
- i** `Path` replaces the old `File` objects in Java.

Path objects

`Path` specifies the location of a file or directory on disk:

- **Key Capabilities:**
 - Check if files or directories exist
 - Create or delete files and directories
 - List files in a directory
 - Access file attributes (size, last modified, etc.)
 - Supports platform-independent paths

OBS: It is modern and flexible alternative to the `File` class.

Path objects can have a absolute or relative path

Absolute Path:

- Starts from the **root directory**.
- Contains the full directory path to the file or folder.
- Example: `/home/user/documents/file.txt` or `C:\Users\User\Documents\file.txt`

Relative Path:

- **Relative** to another directory.
- Often based on the application's starting directory.
- Example: `documents/file.txt` when the base is `/home/user/`

i Absolute paths provide a fixed location, while relative paths are flexible and adapt to the base directory.

Creating a `Path` Object

To create a `Path` object you need to use the method `Path.of()`:

```
Path path = Path.of("example.txt");
```

- This points to a file named `example.txt` in the current directory.

Methods of a `Path` object

- `getFileName()` : Returns just the file name.
- `getParent()` : Returns the parent directory.
- `toAbsolutePath()` : Converts to an absolute path.
- many others

Path methods example

```
Path path = Path.of("example.txt");  
System.out.println("File name: " + path.getFileName());  
System.out.println("Parent directory: " + path.getParent());  
System.out.println("Absolute path: " + path.toAbsolutePath());
```

```
example.txt  
null  
C:/path/to/example.txt
```

Path methods example with folder

```
Path path = Path.of("tmp/example.txt");  
System.out.println("File name: " + path.getFileName());  
System.out.println("Parent directory: " + path.getParent());  
System.out.println("Absolute path: " + path.toAbsolutePath());
```

```
example.txt  
null  
C:/path/to/tmp/example.txt
```

Files class methods

The `Path` objects are usually used as param for methods from the `Files` class

- `Files` class:
 - enable reading, writing, and manipulating files and directories.
 - provides utility methods for performing file and directory operations.
 - designed to work closely with `Path` objects

i `Path` and `Files` offering a modern alternative to legacy `File` operations.

Files class methods

- **File and Directory Management:** Easily create, delete, and list files with `Files` methods:
 - `Files.exists(path) :`
 - `Files.size(path)`
 - `Files.createFile(path) :`
 - `Files.createDirectory(path)`
 - `Files.delete(path)`
 - `Files.list(path) :`
 - ...

Files.exists

Check if a file or directory exists at the `Path`.

```
Path path = Path.of("example.txt");  
System.out.println("example.txt exists:" + Files.exists(path));
```

- Returns `true` if `example.txt` exists, otherwise `false`.

Files.size

Return a long for the number of bytes of the

```
Path path = Path.of("example.txt");  
System.out.println("example.txt size:" + Files.size(path));
```

- Ensure the directory is empty before deleting.

Files.createFile

Create a file at the specified `Path`.

```
Path path = Path.of("newFile.txt");  
Files.createFile(path);
```

- This will create `newFile.txt` in the current directory if it doesn't already exist.
- Check with `Files.exists(path)` to confirm.

Files.createDirectory

Create a directory at the specified `Path`.

```
Path dirPath = Path.of("newDirectory");  
Files.createDirectory(dirPath);
```

- This will create a folder named `newDirectory` in the current location.

Files.delete

Delete a file or directory at the `Path`.

```
Path pathToDelete = Path.of("example.txt");  
Files.delete(pathToDelete);
```

- Removes `example.txt` if it exists.
- Ensure the directory is empty before deleting.

Files.list

Example of List files in a directory with `Files.list` :

```
Path path = Path.of("newDirectory");  
var stream = Files.list(path);  
for (Path file : stream.toList()) {  
    System.out.println(file.getFileName());  
}  
stream.close();
```

- Displays each file and directory entry within `newDirectory` .

different way to indicate directories

Different OS uses different way to indicate directories.

- Windows uses counter slash, e.g., `C:\Users\Documents\file.txt`
- UNIX/Linux (including Macs) uses slash, e.g., `/home/user/file.txt` .

`Files.get` creates platform-independent paths.

- `Files.get(String first, String... )` Combines parts of a path

For example, `C:\Users\Documents\file.txt` can be create as:

```
Path path1 = Paths.get("C:", "Users", "Downloads", "file.txt");
```

URI

- `Files.get(URI uri)` also support uses [Uniform Resource Identifier \(URI\)](#)

For files, the URI define the protocol `file://` which should be followed by the root as `/` and the path using `/` as directory separator.

For example, `C:\Users\Documents\file.txt` can be create as:

```
Path path2 = Paths.get(new URI("file:///C:/Users/user/Documents/file.txt"));
```

i Pay attention there is three `/` in the the above URI.

Reading and Writing Files in Java

So far, we only learn how to represent a file, but not how to read and write one

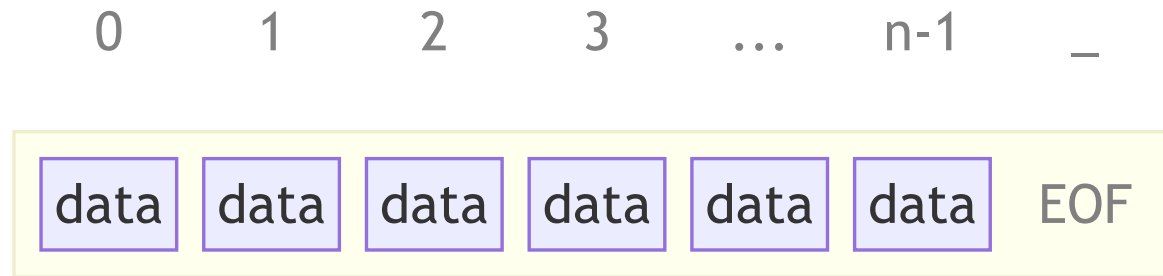
Reading and Writing Files in Java

Java provides various classes to handle file operations.

Java views each file as a sequential stream of bytes.

Every operating system has a mechanism to determine the end of a file.

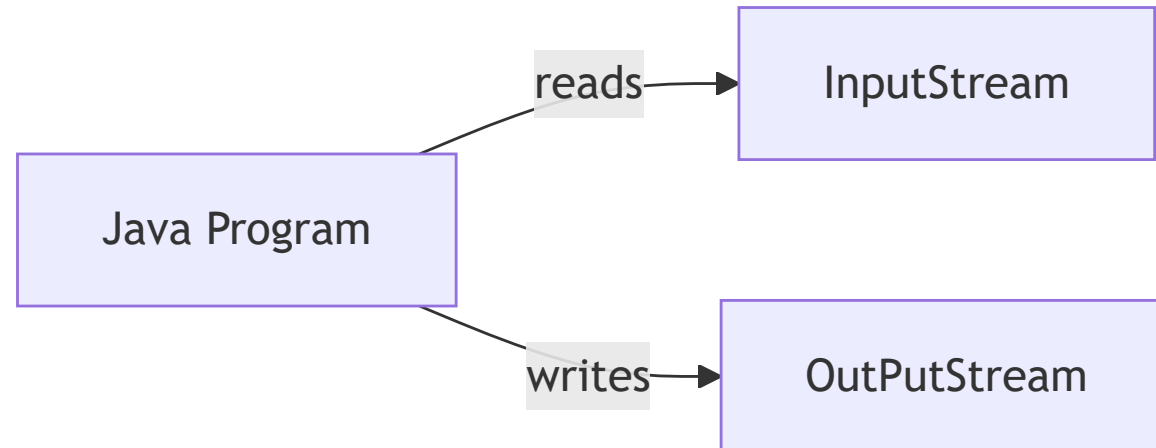
- The most common is an end-of-file marker (EOF)



Reading and Writing Files in Java

Java use the concept of Stream to process Files

- Handle data sequentially.
- Abstracts underlying details of file handling.
 - `InputStream` : Reads data from sources (e.g., files).
 - `OutputStream` : Writes data to destinations (e.g., files).

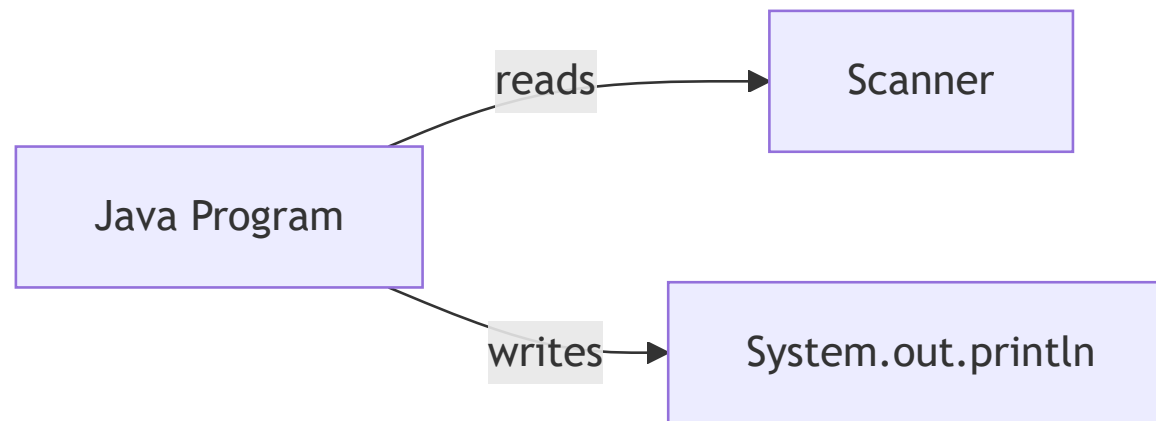


Reading and Writing Files in Java

We already used Java Streams before:

- `Scanner(System.in)` is an example of `InputStream` usage
- `System.out.println()` is an example of `OutputStream` usage

Internally the operation system creates files for each program `System.in` and `System.out`



Reading and Writing Files in Java

There are two ways to reading and writing Files in Java

- **Sequential process files**
 - the `FileWriter`, `Scanner`, and `Formatter` objects
- **Buffered process files**
 - the `BufferedWriter` and `BufferedReader` objects

i We see each of them in next

Sequential process files

- Sequential files **store records in a specific order**, typically by a unique key field.
- Useful for applications that need ordered and persistent data storage.
- Common operations:
 - **Creating** and writing records.
 - **Reading** and retrieving records.
 - **Replacing** all records.
- Ideal for simple, line-by-line text output or input

Writing a Sequential Text File

Example: A Credit Inquiry Program to retrieve and display client data.

- Java does not impose any structure on files by default, so we define custom structures.
- Each record contains an **account number**, **client name**, and **balance**.
- **Account number** serves as the key field for record ordering.

```
100 Bob Blue 24.98  
200 Steve Green -345.67
```

We can use `FileWriter` or `Formatter` for Writing a Sequential Text File

- Their main methods:
 - `write()` and `format` to write, respectively
 - `close()` to finish the writing and release the file

FileWriter

`FileWriter` is an option for writing text data to files, but without built-in formatting.

- Formatting must be manually applied for structured output.

```
FileWriter writer = new FileWriter("clients.txt")
writer.write("100 Bob Blue 24.98\n");
writer.write("200 Steve Green -345.67\n");
writer.close();
```

Formatter

Formatter write text to files with using a format

- Like `System.out.printf`, it allows precise control over data formatting.

```
Formatter output = new Formatter("clients.txt");
int accountNumber;
String firstName;
String lastName;
double balance;
accountNumber=100;
firstName="Blue";
lastName="Blue";
balance=24.98;
output.format("%d %s %s %.2f%n", accountNumber, firstName, lastName, balance);
accountNumber=200;
firstName="Steve";
lastName="Green";
balance=-345.67;
output.format("%d %s %s %.2f%n", accountNumber, firstName, lastName, balance);
output.close();
```

Writing a Sequential Text File

- Uses `Scanner` to read and parse text data from a file.
 - It reads line-by-line and parses each field according to expected data types.
 - `hasNext()` to check if is no the EOF
 - `nextLine()` to get the entire next line

The code below prints lines from the `input.txt` file.

```
Scanner file = new Scanner(Path.of("input.txt"));
while (file.hasNext()) {
    String line = file.nextLine();
    System.out.println(line);
}
```

Example `next()` `nextInt()`, `nextDouble()`

```
100 Bob Blue 24.98
200 Steve Green -345.67
```

For the `clients.txt`, the code below will use

- `nextInt()` to get a id (int)
- `next()` to get a each part of the name
- `nextDouble()` to get a double.

```
Scanner input = new Scanner(Path.of("clients.txt"))
while (input.hasNext()) {
    int account = input.nextInt();
    String firstName = input.next();
    String lastName = input.next();
    double balance = input.nextDouble();
    System.out.printf("account %d from %s %s has balance %.2f%n",
        account, firstName, lastName, balance);
}
```

in-class quiz 1

Why `scanner.nextLine()` in the program below does not get "Hello, World!" from the file "output.txt"?

```
var path = Path.of("output.txt");
var formatter = new Formatter(path.toFile());
formatter.format("Hello, %s!", "World");

var scanner = new Scanner(path);
while (scanner.hasNextLine()) {
    System.out.println(scanner.nextLine());
}
```

- Because `Path.of()` does not create a new file
- Because `Formatter` only works with console output
- Because the `Formatter` was not closed, so data wasn't flushed to the file
- Because `Scanner` requires a `BufferedReader`

Sequential Text File

Pay attention that:

- both `FileWriter` or `Formatter` require a call their `close()` method to finish the writing
- if the file exists, `FileWriter` or `Formatter` replace all previous content
- **Updating a sequential file typically** involves reading the existing data, modifying the necessary parts, and writing the data back into a new file.

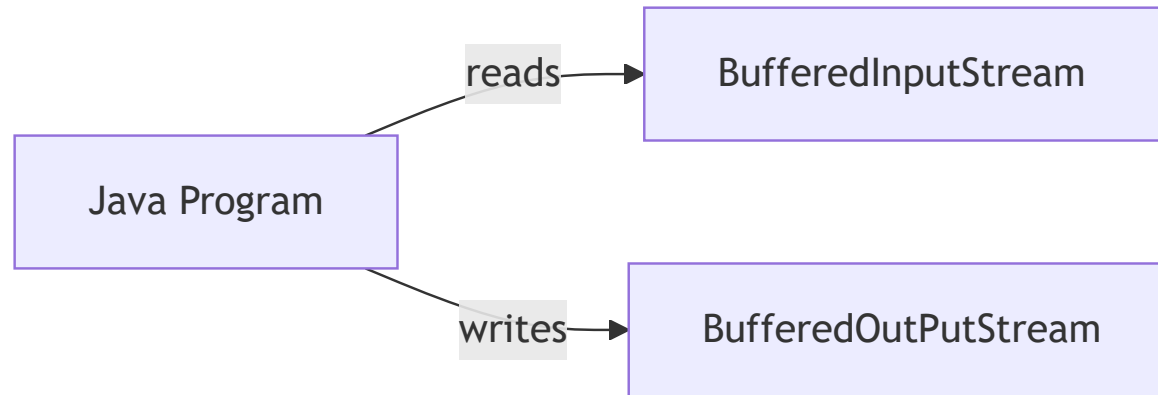
The Sequential files is not optimised for long size files

Buffered Text files

- Improve performance by reducing the number of I/O operations.
- They use **Buffered Streams**, which are efficient for I/O operations.
 - Instead of accessing the disk directly each time data is read or written, data is stored temporarily in memory.
 - This minimizes the time spent accessing the file system.
- **Ideal scenarios**
 - Working with **large files** where direct read/write operations would be slow.
 - Performing **frequent read/write operations**, like logging, where small amounts of data are written continuously.

Streams vs Buffered Streams

- You need to use the `BufferedInputStream` and `BufferedOutputStream`
 - Similar to `FileInputStream` and `FileOutputStream` but with buffer



BufferedWriter

- You can one using `Files.newBufferedWriter(path)`
- Its main function is `write()` and `close()`

```
var path = Path.of("example.txt");  
  
var writer = Files.newBufferedWriter(path);  
writer.write("Hello, World!\n");  
writer.write("Welcome to Java File I/O with buffering.");  
  
writer.close();
```

BufferedReader

- You can one using `Files.newBufferedReader(path)`
- Its main function is `readLine()` and `close()`

```
var path = Path.of("example.txt");
var reader = Files.newBufferedReader(path);

var line = reader.readLine();
while (line != null) {
    System.out.println(line);
    line = reader.readLine();
}

reader.close();
```

in-class quiz 2

Which of the following statements about `BufferedWriter` and `Formatter` is true?

- `BufferedWriter` formats text automatically, while `Formatter` provides low-level text writing.
- `BufferedWriter` is ideal for large files, while `Formatter` is ideal for formatting complex output.
- Both `BufferedWriter` and `Formatter` are used only for file reading.
- Both classes are used only with `Path` and `Scanner`.

Introduction to Collections

What a Collections?

- It is a container of objects
- It provides **efficient methods** manipulate data without knowledge of how the object is being stored.
- Commonly used collections include:
 - **List**: elements have an order
 - **Set**: Ensures unique elements, ideal for eliminating duplicates.

What a Collections?

- It is a container of objects
- It provides **efficient methods** manipulate data without knowledge of how the its is being stored.
- Commonly used collections include:
 - **List:**
 - **ArrayList:** Best for frequent access by index, resizable array.
 - **LinkedList:** Doubly linked list, efficient for frequent insertions/deletions.
 - ...
 - **Set:**
 - **HashSet**
 - ...

The Diamond notation (<>)

- Since any object can be in a container
- If not expressed, the compiler will not know the element type and consider each element as a generic Object
- The diamond (<>) notation tells the compiler to determine what belongs in the angle brackets.

```
ArrayList<String> sitems = new ArrayList<String>(); // ArrayList of String  
ArrayList<Integer> items = new ArrayList<Integer>(); // ArrayList of String  
ArrayList<Double> ditems = new ArrayList<Double>(); // ArrayList of Double
```

ArrayList

- The collection class ArrayList

Characteristics

- Resizable array implementation of the List interface.
- Allows duplicate elements and maintains the order.
- **Fast access**, slower inserts unless at the end.

Common Methods

- `add()` , `get()` , `remove()`

Example of ArrayList

Adding and Accessing Elements

```
ArrayList<String> fruits = new ArrayList<String>();  
fruits.add("Apple");  
fruits.add("Banana");  
fruits.add("Cherry");  
  
System.out.println("Fruits: " + fruits);  
fruits.remove("Banana");  
System.out.println("After removal: " + fruits);  
  
String fruit = fruits.get(1);  
System.out.println("Fruit at index 1: " + fruit);
```

Reading Data from a Sequential Text File to a ArrayList

```
ArrayList<String> records = new ArrayList<String>();  
Scanner file = new Scanner(Path.of("clients.txt"));  
while (file.hasNextLine()) {  
    String line = file.nextLine();  
    records.add(line);  
}  
file.close();
```

LinkedList

- The collection class LinkedList

Characteristics

- It is a doubly-linked list that implementation is `List` and `Deque` interfaces.
- Efficient for adding and removing at the beginning and end.
- **Faster inserts/removals**, slower access.

Common Methods

- `addFirst ()`, `addLast ()`, `removeFirst ()`, `removeLast ()`

Example of LinkedList

Adding and Accessing Elements

```
LinkedList<String> fruits = new LinkedList<String>();  
fruits.add("Apple");  
fruits.add("Banana");  
fruits.add("Cherry");  
  
System.out.println("Fruits: " + fruits);  
  
fruits.addFirst("Mango");  
System.out.println("After adding at first: " + fruits);  
  
fruits.removeLast();  
System.out.println("After removing last: " + fruits);
```

Example of HashSet

```
HashSet<String> set = new HashSet<String>();  
set.add("Dog");  
set.add("Cat");  
set.add("Bird");  
set.add("Dog"); // Duplicate, won't be added
```

var Type in Java (Introduced in Java 10)

var keyword allows local variable type inference, making code cleaner and reducing redundancy

- infer the variable type from the assigned value.
- you only write type once

```
// Without `var`  
ArrayList<String> alist = new ArrayList<String>();  
LinkedList<String> llist = new LinkedList<String>();  
HashSet<String> set = new HashSet<String>();  
  
// With `var`  
var alist = new ArrayList<String>();  
var llist = new LinkedList<String>();  
var set = new HashSet<String>();
```

in-class quiz 3

What will the following program output, given the handling of duplicates in `ArrayList` and `Set`?

```
var list = new ArrayList<String>();  
list.add("apple"); list.add("banana"); list.add("apple"); list.add("banana");  
  
var set = new HashSet<String>(list);  
set.add("apple"); set.add("banana"); set.add("cherry"); set.add("apple");  
  
System.out.println("ArrayList: " + list);  
System.out.println("HashSet: " + set);
```

- ArrayList: [apple, banana, apple, banana], HashSet: [apple, banana, cherry]
- ArrayList: [apple, banana, apple, banana], HashSet: [apple, cherry, banana, apple]
- ArrayList: [apple, banana, apple, banana], HashSet: [apple, banana, apple, cherry]
- ArrayList: [apple, banana], HashSet: [apple, banana, cherry]

When to Use Each?

- ArrayList is better for storing and accessing data.
 - Use when frequent access by index.
- LinkedList is better for data manipulation.
 - Use when frequent insertion/removal.
- HashSet
 - Use when require unique elements

Collections Class Functions

helpful methods for working with collections are:

sort(List list)

```
var list = new ArrayList<Integer>(List.of(3, 1, 4, 1, 5));  
Collections.sort(list); // Result: [1, 1, 3, 4, 5]
```

reverse(List list)

```
Collections.reverse(list); // Result: [5, 4, 3, 1, 1]
```

shuffle(List list)

```
Collections.shuffle(list); // Result: [random order of elements]
```

min(Collection coll) / max(Collection coll)

```
int min = Collections.min(list); // Result: 1  
int max = Collections.max(list); // Result: 5
```

Final remarks

Review our Learning Goals

What is file I/O?

- File I/O is fundamental for data-driven applications.

How to do file I/O in Java?

- For sequential text file operations, use `FileWriter`, `Formatter` and `Scanner`
- For buffered text file operations, use `BufferedReader`, and `BufferedWriter`.

What are Java Collections?

- Data structures which offer more support than arrays
 - e.g., they increase size dynamically and functions
- `ArrayList`, `LinkedList` and `Set` collections
- `Collections` has useful functions, e.g. `Collections.sort()`

DEPARTMENT OF COMPUTER SCIENCE



Thank you! 😊

Do you have any questions 🙋 ?

next lecture will cover "Top-down design"